KRISNA PRANAV

Senior Software Engineer · AI/ML | Distributed Systems · Cloud-Native Platforms · Applied AI | Python · Go · TypeScript

Singapore | krisna.pranav@gmail.com | linkedin.com/in/krishpranav | github.com/krishpranav | krishpranav.github.io

SUMMARY

Software engineer with **8+ years** shipping production systems and **4+ years in AI/ML**. I build the platform side of applied AI: distributed backends, APIs, and cloud-native services on AWS and Azure, plus the ingestion, retrieval, and agent pipelines (**LangGraph, LangChain, Claude/OpenAI**) on top. I've built RAG over legal and compliance content that re-indexes as new rulings land, a streaming LLM gateway tuned for time-to-first-token, and a zero-downtime C++ to Go migration on a PayPal tier serving 15 to 20M requests/day. I treat CI/CD, tests, evals, and tracing as part of the feature, and write code others can build on.

TECHNICAL SKILLS

Distributed Systems:	Service architecture, concurrency & backpressure, async I/O (asyncio, Tokio), caching, streaming (WebSockets / SSE), zero-downtime migrations, latency & throughput profiling
AI & Agents:	LangGraph, LangChain, Anthropic Claude & OpenAI APIs, tool calling, multi-agent orchestration, RAG, hybrid retrieval & reranking, embeddings, vector search (FAISS, HNSW)
MLOps & Evals:	Model evaluation, LLM tracing (Langfuse), monitoring, inference tuning, containerized deploys
Cloud & DevOps:	AWS, Azure (AKS / ACI), Docker, Kubernetes, CI/CD (GitHub Actions), automated testing, deployment automation, serverless / edge functions
Backend & Data:	FastAPI, gRPC, REST API design, PostgreSQL, Redis, Neo4j, schema & index design, caching, read-heavy data access patterns
Languages & Web:	Python, Go, TypeScript, Rust, C/C++, SQL · React, Next.js, Node.js

EXPERIENCE

AI/ML Engineer, Turing — Present

Remote; agentic AI systems, LLM orchestration, and production AI infrastructure

- Build agentic workflows and LLM orchestration on Anthropic models: multi-step reasoning, tool calling, and durable state that lets long-running agents resume after a failure instead of starting over.
- Design the gRPC/REST layer that routes requests to agents, runs tasks concurrently, and defines the data contracts between services, so new workflows plug in without changes to the core.
- Write latency-sensitive infrastructure in Rust (async/Tokio) and C++, bridged to Python through PyO3/FFI for model integration and inference serving.
- Evaluate model outputs against rubrics and reference solutions, and own inference tuning, observability, and deployment for frontier-model work delivered to Anthropic, OpenAI, Google, and Meta.

AI/ML Engineer, Crossing Hurdles — Mar 2026 to Sep 2026

Concurrent contract, remote; forward deployed on production AI for clients including Handshake, Otter, and Snorkel

- Worked inside client engineering teams (Handshake, Otter, Snorkel) and closed every scoped task on their real-time inference and orchestration paths, in Python, Rust, C++, and TypeScript.
- Built a streaming inference gateway that fans out concurrent LLM calls and streams tokens back over WebSockets/SSE, keeping time-to-first-token low under concurrent load.
- Added an in-memory embedding cache and incremental re-indexing, so retrieval stayed fast as traffic grew and content updates no longer triggered a full re-index.

Founding Engineer (#1), Stealth · VC-backed AI Notetaker — Jan 2025 to Jul 2026

Privacy-first AI meeting assistant, built 0 to 1 as the first engineering hire

- Made the architecture calls and shipped the product from the meeting bot to the user-facing app: Python/FastAPI services on PostgreSQL, Dockerized and deployed on AWS.
- Built a bot that joins Zoom, Microsoft Teams, Google Meet, and phone (PSTN) calls on its own, on a shared SDK layer over each platform's APIs. Supporting a new platform meant writing one adapter, not another bot.
- Live transcription streams into LangGraph agents that produce structured summaries, action items, and decisions, with sensitive meeting data kept under the user's control.
- Added RAG over each user's meeting history and wired Langfuse tracing into every LLM call, so latency, token cost, and bad outputs could be traced to a specific prompt and step.
- Set up CI/CD, the automated test suite, and code review conventions, mentored the engineers who joined after me, and worked directly with early users on what to build next.

AI/ML Engineer, Uber **Mar 2024 to Aug 2026**

Applied ML on production systems at scale

- Built and integrated model-driven services that make real-time decisions on high-traffic surfaces, working to tight latency budgets and reliability targets.
- Worked on the data and inference pipelines behind ML features and the Go/Python backends that serve predictions, alongside data science, infrastructure, and product teams.

AI/ML Engineer, Legal RAG Platform (Contract) **Jun 2021 to Jan 2022**

Remote; retrieval-augmented legal and compliance information system

- Built a production RAG system that answers procedural, compliance, and legal questions from a curated knowledge base. Every answer cites its source passage, so reviewers can check it.
- Designed the content ingestion pipeline: PDF parsing, semantic chunking, metadata tagging (source, jurisdiction, document type), and embedding upsert into a hybrid vector + keyword store with reranking. Scanned pages went through vision-LLM extraction into the same pipeline.
- Built a scheduled monitoring agent that watches designated sources, detects new rulings, and re-indexes the live knowledge base, which ended manual upkeep of the corpus.

Senior Rust Engineer, Aether (AI Desktop Agent) **Sep 2020 to May 2021**

Contract, on-device retrieval & vector search

- Built a privacy-first desktop assistant in Rust: a local semantic engine indexes files, embeds them on-device, and answers context-aware queries with low latency.
- Designed the on-device HNSW vector index for strong recall on large corpora without a GPU, and owned packaging and release of the macOS/Linux binaries.

Senior Software Engineer, PayPal (External Contract) **Aug 2019 to Jul 2020**

High-throughput backend migration

- Migrated critical C++ services to Go on a tier handling **15 to 20 million requests/day**, matching the legacy system's p99 latency while simplifying deploys.
- Built transaction-processing concurrency on goroutines with bounded channels for backpressure, and ran the cutover with zero downtime by shifting traffic over gradually.

Earlier roles (2017 to 2019): a low-latency on-chain trading platform (Rust, Next.js/TypeScript), full-stack web (Next.js, React, Node.js, Python), and production mobile and blockchain apps (Flutter, React Native, Solidity).

SELECTED OPEN-SOURCE WORK

Tauri (github.com/tauri-apps/tauri): Rust framework for cross-platform desktop apps. Fixed a memory leak in the core runtime.

- In **tauri-runtime-wry**, `WithWebview` leaked Objective-C retains on macOS/iOS. Replaced `Retained::into_raw()` with scoped `Retained::as_ptr()` bindings so retain/release stays balanced and WebContent processes are freed; RSS held flat over a 3-minute stress loop. PR #15224, merged upstream.

Playwright (github.com/microsoft/playwright): Microsoft's cross-browser automation and testing framework. Fixed a browser crash in the Chromium driver.

- Navigating to privileged hosts like `chrome://extensions` from a non-persistent CDP context segfaulted Chromium. Guarded `page.goto()` in `crPage.ts` with a per-browser blocklist and an actionable error, turning a session-killing crash into a recoverable one; 7,800+ tests passing. PR #41986, merged upstream.

EDUCATION

BE, Computer Science Engineering, MCET